

Here's a breakdown of how each namespace contributes.

- **PID namespace:** Provides separate process ID trees, so each container believes it has its own PID space.
- **NET namespace:** Gives each container its own network interfaces, routes, and firewall rules.
- **MNT namespace:** Isolates the filesystem, allowing each container to have its own directory structure.
- **IPC namespace:** Manages inter-process communication isolation, preventing containers from accessing each other's message queues and semaphores.
- **UTS namespace:** Offers hostname and domain isolation, making each container feel like it's running on a unique machine.
- **USER namespace:** Maps user IDs from the host to container-specific ranges, potentially offering a layer of privilege isolation.

Here's a simple example of creating a Docker container with user namespace isolation enabled.

```
docker run --rm -it --userns=remap
busybox
```

While namespaces do a decent job of separating resources, they don't offer airtight isolation. For example, if a container exploits a kernel vulnerability, it could potentially access other namespaces or even the host system. This risk makes namespaces inherently weaker than the isolation provided by hypervisors in virtual machines.

Control groups (cgroups)

Cgroups limit and account for a container's resource usage (CPU, memory, I/O, etc). This ensures that a container can't hog system resources, providing stability in multi-container environments. For instance, you can limit a container's memory the following way.

```
docker run -m 512m my-container
```

However, cgroups aren't designed to enhance security. They are more about controlling resource usage. If a container is compromised, cgroups won't prevent it from attempting to exploit the shared kernel or other containers.

A cross-comparison with traditional virtualisation

I know that many people criticise Docker for the elements of a traditional virtual machine (VM) that it is missing. But that's the thing — Docker's not trying to be a VM! Virtual machines operate with a hypervisor sitting directly on the hardware, creating completely independent environments. Each VM has its own dedicated OS, kernel, and everything else, which means they're genuinely isolated from one another. Even if you manage to break into one VM, you're still miles away from the others. That's security in the real sense.

With Docker, you've got containers that run on a shared OS kernel. There's no hypervisor acting as a guard and no full OS instance per container. They're just processes running on the same machine, isolated by Linux namespaces and cgroups.

Here's an example to put it in perspective. In a VM, the process of creating an isolated environment might look something like this:

```
virt-install \
  --name=my-vm \
  --ram=2048 \
  --disk path=/var/lib/libvirt/images/
my-vm.qcow2,size=10 \
  --vcpus=2 \
  --os-type=linux
```

That's a full-blown instance with its own kernel. Now, in Docker, the equivalent setup is as simple as the following example.

```
docker run -d --name=my-container nginx
```

Sure, that container is up and running in seconds, but it's still using

the host's kernel, sharing it with every other container on the system. That's why Docker's isolation isn't quite what it seems.

Security concerns

When you share a kernel, you increase the attack surface. If a container finds a way to exploit that shared kernel, it's game over. That's not me being dramatic; this has happened before with container escape vulnerabilities like CVE-2019-5736, where attackers broke out of the container and gained root access on the host. It's these kinds of risks that VMs don't have to worry about because each one is running its own kernel.

You may argue that tools like AppArmor, SELinux, and seccomp profiles can mitigate these risks in Docker. But let's be real — adding layers to compensate for a fundamentally shared architecture isn't the same as having true isolation. When all containers are tied to the same kernel, any vulnerability there becomes an open door to all containers on that host. Here's a quick-and-dirty snippet showing how a container breakout might look using a chroot escape technique, for example.

```
mkdir /host
mount --bind / /host
chroot /host
```

What's happening here? We're remounting the host filesystem within the container's process space, effectively bypassing the container's isolation. This is an example of why Docker's kernel sharing is fundamentally dangerous. In a VM, this would never fly.

By default, Docker containers run as *root*. Is that a good thing? That's a whole different debate, but for the sake of this topic, no it's not. And while you can mitigate this by running containers as *non-root* users, many developers skip that step (I, too, do this) because we are lazy. When you run as *root*, and an attacker compromises your container, they're not just in the front door; they've got the keys